

Foundational Verification of Probabilistic Programs

Mechanising the Lilac probabilistic separation logic in Lean

Edwin Fernando

June 18, 2026

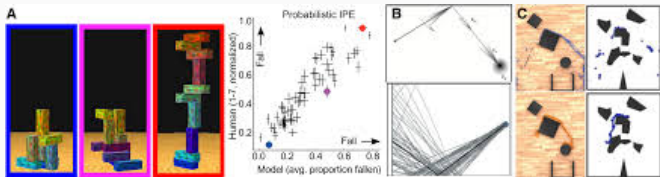
Department of Computing, Imperial College London

Probabilistic Programming is Everywhere

Probabilistic programming is everywhere

The most general feature of a PPL — sampling from a distribution — lives in the standard library of every major language.

- **Cryptography** — protocols, ZK proofs
- **Differential privacy**
- **Randomised algorithms**
- **Distributed systems**
- **Bayesian inference** & probabilistic ML
- **Scientific simulation** — climate, physics
- **Robotics & perception** — intuitive physics



Discrete vs. continuous

The dividing line that matters runs through the **applications**.

Discrete

Coin flips, Bernoulli / categorical choices, dice.

Cryptography, differential privacy, randomised algorithms — Applications traditionally within CS - much larger focus.

Continuous

Gaussian noise, sensor models, Bayesian priors over $\mathbb{R}$, physical quantities.

Scientific simulation, Bayesian inference, ML — where the **real-world modelling** happens.

Continuous distributions are where prior verification work fell short — and where this project makes its **first mechanised** contribution.

Verifying probabilistic programs is notoriously hard

- Conventional **testing fails**: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole **distribution**, not of any single run.
- Reasoning about **independence** and conditioning is subtle and error-prone by hand.

Verifying probabilistic programs is notoriously hard

- Conventional **testing fails**: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole **distribution**, not of any single run.
- Reasoning about **independence** and conditioning is subtle and error-prone by hand.

So we need proof

Machine-checked, **foundational** arguments about distributions — resting on nothing but the axioms of mathematics.

The first **mechanisation** of **foundational verification**
for **probabilistic programs**
supporting continuous distributions.

1. Soundness proofs
2. Interactive *Proof Mode*
using Iris

The first **mechanisation** of **foundational verification**
for **probabilistic programs**
supporting continuous distributions.



Contribution

1. Soundness proofs
2. Interactive *Proof Mode* using Iris

Using a theorem prover
(Lean/Rocq/Agda)
Assuming only axioms of math¹



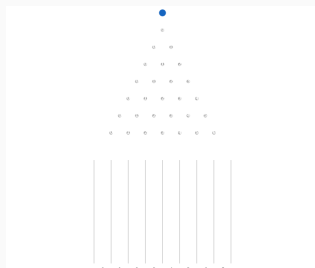
The first **mechanisation** of **foundational verification**
for **probabilistic programs**
supporting continuous distributions.

¹Background Sec 2.4.1 “Dependent Type Theory”

A program is a distribution over runs

$$\text{flip} : (m : \mathbb{R}) \rightarrow \text{Distrib } \{m, m + 1\}$$

an individual “run” of the
program



Honing in on a single
marble

$X_0 \leftarrow \text{flip } 0$

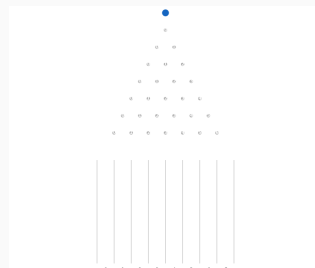
$X_1 \leftarrow \text{flip } X_0$

$X_2 \leftarrow \text{flip } X_1$

ret X_2

|||

The program as a whole



All the marbles at once

for $(3, \text{flip } 0, \lambda X. \text{flip } X)$

$$\begin{array}{l} X_0 \leftarrow \text{flip } 0 \\ X_1 \leftarrow \text{flip } X_0 \\ X_2 \leftarrow \text{flip } X_1 \\ \text{ret } X_2 \end{array} \quad \equiv \quad M : \quad \llbracket \Gamma \rrbracket \rightarrow \text{Measure } ty$$

Refining the domain: the Hilbert cube

$X_0 \leftarrow \text{flip } 0$

$X_1 \leftarrow \text{flip } X_0$

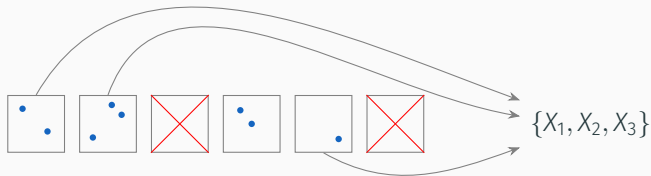
$X_2 \leftarrow \text{flip } X_1$

ret X_2

$\equiv$

$M : \text{HC} \rightarrow \llbracket \Gamma \rrbracket \rightarrow \text{Measure ty}$

$\text{HC} = \text{Hilbert cube} = [0, 1]^{\mathbb{N}}$



How to verify: probabilistic separation logic

$$\underbrace{X \leftarrow \text{unif}[0, 1]; \text{ ret } X}_{\text{program}} \\ \equiv \\ \underbrace{\lambda E \mapsto \int (\lambda \mu \mapsto \mu E) d(\text{map } (\lambda X \mapsto \text{dirac } X) \text{ unif}[0, 1])}_{\text{denotation}}$$

How to verify: probabilistic separation logic

$X \leftarrow \text{unif}[0, 1]; \text{ ret } X$
program

$\equiv$

~~$\lambda E \mapsto \int (\lambda \mu \mapsto \mu E) d(\text{map}(\lambda X \mapsto \text{dirac } X) \text{ unif}[0, 1])$~~
denotation

How to verify: probabilistic separation logic

$X \leftarrow \text{unif}[0, 1]; \text{ ret } X$
program

$\equiv$

~~$\lambda E \mapsto \int (\lambda \mu \mapsto \mu E) d(\text{map}(\lambda X \mapsto \text{dirac } X) \text{ unif}[0, 1])$~~
denotation

reason directly about the variables

$X \sim \text{unif}[0, 1]$

Iris, and the logic of bunched implications (BI)

The separation logic is built on the Lean port of [Iris](#); its assertions `Prop` form a **BI (bunched-implication) logic**.

$\ulcorner, \urcorner : \text{Prop} \rightarrow \text{Prop}$	pure embedding
<code>True</code> , <code>False</code> , <code>emp</code> : <code>Prop</code>	units
$\wedge, \vee, \Rightarrow, *, \multimap : \text{Prop} \times \text{Prop} \rightarrow \text{Prop}$	binary connectives
$\forall, \exists : \forall A. (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$	quantifiers

$*$ separating conjunction $\multimap$ magic wand — the separation-logic-specific connectives.

Iris, and the logic of bunched implications (BI)

Assertions `Prop` form a **BI (bunched-implication) logic** on the Lean port of **Iris**.

$\ulcorner \cdot \urcorner : \text{Prop} \rightarrow \text{Prop}$	pure embedding
<code>True</code> , <code>False</code> , <code>emp</code> : <code>Prop</code>	units
$\wedge, \vee, \Rightarrow, *, \multimap : \text{Prop} \times \text{Prop} \rightarrow \text{Prop}$	binary connectives
$\forall, \exists : \forall A. (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$	quantifiers

Iris, and the logic of bunched implications (BI)

Assertions `Prop` form a **BI (bunched-implication) logic** on the Lean port of **Iris**.

$\ulcorner \cdot \urcorner : \text{Prop} \rightarrow \text{Prop}$	pure embedding
<code>True</code> , <code>False</code> , <code>emp</code> : <code>Prop</code>	units
$\wedge, \vee, \Rightarrow, *, \multimap : \text{Prop} \times \text{Prop} \rightarrow \text{Prop}$	binary connectives
$\forall, \exists : \forall A. (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$	quantifiers

- **A logic inside a logic.** `Prop` is an object logic inside Lean's meta-logic.²

Iris, and the logic of bunched implications (BI)

Assertions `Prop` form a **BI (bunched-implication) logic** on the Lean port of *Iris*.

$\ulcorner, \urcorner : \text{Prop} \rightarrow \text{Prop}$	pure embedding
<code>True, False, emp</code> : <code>Prop</code>	units
$\wedge, \vee, \Rightarrow, *, -* : \text{Prop} \times \text{Prop} \rightarrow \text{Prop}$	binary connectives
$\forall, \exists : \forall A. (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$	quantifiers

- **A logic inside a logic.** `Prop` is an object logic inside Lean's meta-logic.²
- **Impredicativity.** $\forall/\exists$ range over an *arbitrary* type A — including `Prop` itself: we may quantify over propositions.

²Revisited later in the talk.

Mechanisation of Core³ Lilac in Lean

a **continuous** probabilistic separation logic with **separation as independence**.

Along the way, plausibly the first (or second) instantiation of **Iris-lean** for a program logic of this kind.

³Extended Lilac adds reasoning about conditioning.

The formalisation, in one picture

